

TECHPATH

Learning Path Intelligence Actor

Apify × She Code Africa Hackathon
PRD + Concept Note + Actor Architecture

Core idea

Turn the scattered technical-learning web into a structured, personalized path from beginner to job-ready.

This document reframes the original idea around what an Apify Actor is actually good at: a reusable cloud tool that accepts structured input, performs a defined web/data job, and returns structured output. Apify documents Actors as serverless programs for web scraping, browser automation, data processing and AI-agent workflows. [\[cite\]](#)[\[turn0search0\]](#)[\[turn0search1\]](#)

Recommended Actor name: TechPath Learning Intelligence

Store slug suggestion: techpath-learning-intelligence

Prepared as a hackathon concept

1. Executive Summary

TechPath is not another course platform. It is a reusable Apify Actor that collects and analyzes scattered technical-learning resources from the web and turns them into an ordered learning plan based on a learner's goal, current skills and constraints.

A learner may find a Python course, a DSA playlist, a machine-learning bootcamp, a LangChain tutorial, a PyTorch course, a system-design video and a random GitHub project, but still have no clear answer to the most important question: what should I learn next, and why?

TechPath addresses the information-structure problem. It discovers resources, extracts the skills they teach, estimates prerequisites, groups resources by topic, identifies gaps, generates assessments, recommends projects, and finds relevant hackathons or other practical opportunities. The output is structured data that can power an app, AI agent, dashboard, career coach or learning assistant.

The Actor is designed as infrastructure rather than a consumer-only app. This aligns with Apify's guidance that Actors are especially suitable for isolated, repeatable backend automation jobs with well-defined inputs and outputs. [\[cite\]turn0search1](#)

One-line pitch

“TechPath turns the scattered internet of tech education into a structured path that tells you what to learn, what you are missing, what to build, and what real opportunities can help you prove the skill.”

What makes it different

- It is not simply a course recommender; it builds a dependency-aware learning path.
- It can identify missing foundations before recommending advanced topics.
- It combines learning resources with assessments and practical projects.
- It can connect the learner's target skill to relevant hackathons, competitions, internships and other opportunities.
- It returns reusable structured data that other developers and AI applications can consume.

2. Problem Statement

There is an abundance of technical learning content, but very little structure connecting it into a coherent progression.

Someone starting in software engineering, data science or machine learning can access thousands of videos, courses, documentation pages, GitHub repositories and communities. The problem is not access; it is sequencing, depth, relevance and feedback.

A beginner can jump from Python to machine learning, then to LangChain or agents because those topics are trending, without knowing whether they have the mathematical, programming, data or software-engineering foundations required. Another learner may spend weeks watching tutorials without building anything or testing whether they actually understand the material.

At the same time, practical growth opportunities such as hackathons, coding competitions, internships and open-source projects are also scattered across the web. The learner has to manually search for opportunities that match their current skill level and the skills they are trying to build.

The result is fragmented learning: lots of content, little direction; lots of watching, little assessment; and many opportunities, but poor matching.

Why this matters in African communities

For many African learners, especially students and early-career developers, the internet is a major source of technical education. The learner may not have a personal mentor who can say what to learn first, what to skip, what project to build next, or which opportunity fits their current level. TechPath aims to turn publicly available learning information into a clearer self-guided pathway.

3. Product Vision

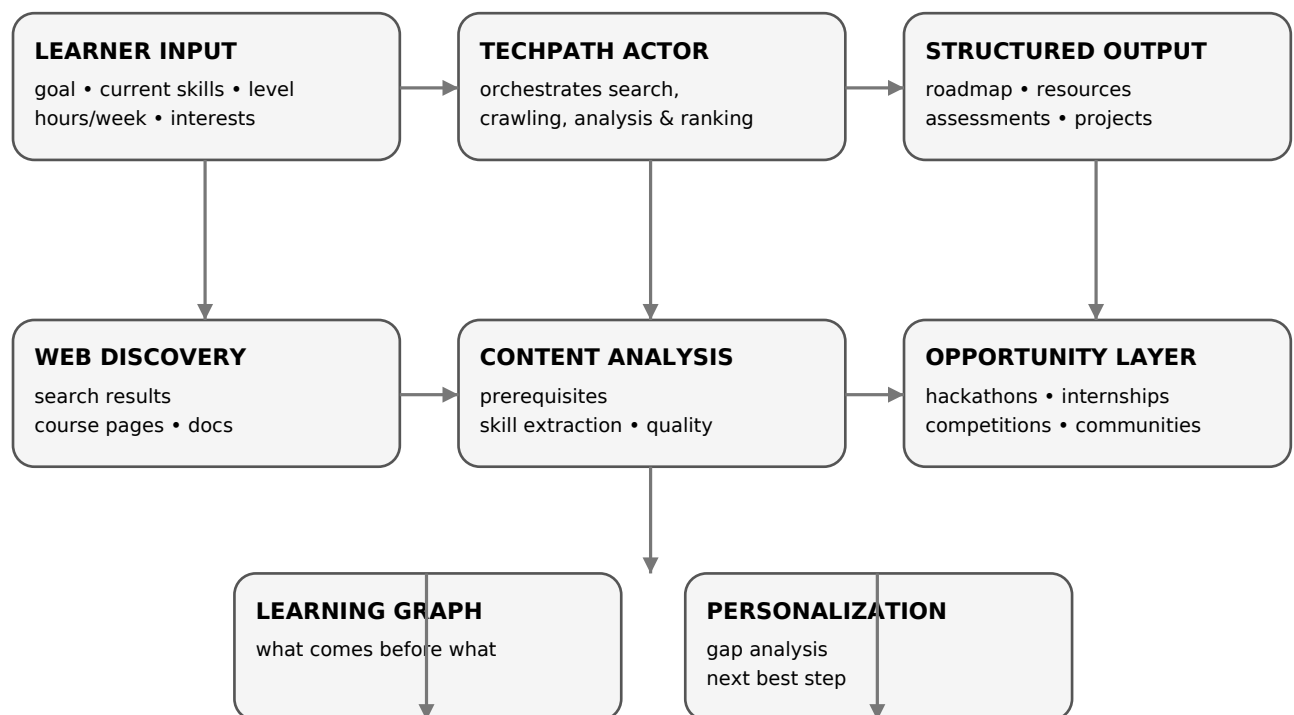
Vision: Make technical learning navigable rather than overwhelming.

Mission: Help a learner move from “I want to become X” to “I know what I need to learn next, I can prove that I learned it, and I know what to build or apply for next.”

The system should support multiple technical paths rather than locking itself to machine learning. Example paths include Software Engineering, Machine Learning Engineering, Data Science, Data Engineering, Cybersecurity, Cloud/DevOps and AI Engineering.

4. Product Concept

TechPath has two layers: the Actor and the application experience. The Actor is the core hackathon deliverable. A future frontend can call the Actor and present the results to learners.

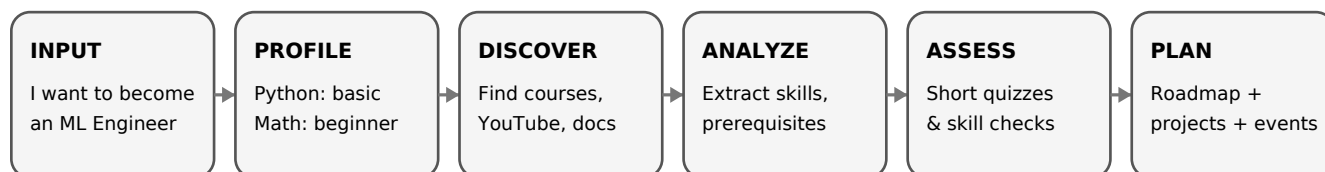


The Actor accepts a learner profile and target. It discovers relevant web resources, analyzes their content and relationships, then returns a structured learning plan. It can also produce assessments, project briefs and opportunity matches.

5. Target Users

- Beginner: knows little about the field and needs foundations and sequencing.
- Intermediate learner: has completed some courses but does not know what to learn next.
- Career switcher: has transferable skills but needs a structured bridge into a technical role.
- Student preparing for interviews: needs DSA, CS fundamentals, projects and assessment practice.
- Builder: wants projects or hackathons that strengthen a particular skill.
- Educator/mentor: wants a starting point for personalized learning plans.

6. Core User Experience



Example input

```
{ "goal": "Machine Learning Engineer", "current_level": "Beginner", "known_skills": ["Python basics"], "hours_per_week": 8, "interests": ["NLP", "healthcare AI"], "location": "Nigeria" }
```

The Actor should not simply return a list of courses. It should return a reasoned progression and explain why each stage appears before the next.

7. Main Features

A. Learning Path Builder

Builds a staged roadmap from foundations to the target role. The path is based on extracted prerequisites and the learner's current skills, not just popularity.

B. Skill Gap Analysis

Compares the learner's known skills with the skills implied by the target role and selected resources. It flags missing prerequisites.

C. Resource Discovery

Finds relevant courses, YouTube videos/playlists, documentation, tutorials, books, repositories and project guides. The Actor can use Apify search/crawling tools to collect current public web information.

D. Assessments

Generates short skill checks for each stage: multiple-choice questions, conceptual questions, coding prompts or mini tasks. The assessment should determine whether the learner is ready to move forward.

E. Project Recommendation

For every major stage, recommends a practical project. At the target level, it produces a larger capstone project with a problem statement, objectives, suggested stack, milestones and evaluation criteria.

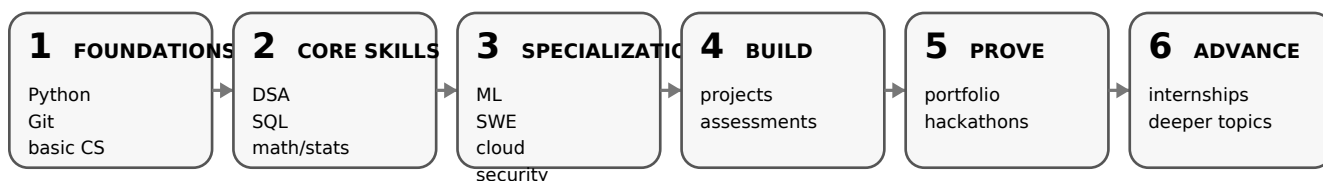
F. Opportunity Matching

Finds hackathons, competitions, internships, open-source opportunities or communities relevant to the learner's current skill and target niche. The goal is not to show every opportunity; it is to match opportunities to the learner's growth stage.

G. Progress-aware next step

After an assessment, the next run can use the result to adjust the path: continue, review a prerequisite, or move forward.

8. Learning Path Logic



The Actor does not claim there is one universal path; it builds a justified path from prerequisites, learner state and target role. It can also identify missing foundations before recommending advanced topics such as LLMs, agents or MLOps.

The important design principle is prerequisites before trends. A learner interested in LLMs should not automatically be sent to LangChain, RAG or agents if their Python, data structures, statistics, machine-learning or deep-learning foundations are weak. The system can still show advanced topics, but it should label them as later-stage material when prerequisites are missing.

9. Assessment Design

Assessments make TechPath different from a static resource directory. A resource tells the learner what to watch; an assessment checks whether the learner can use what they watched.

Stage	Assessment	Example
Python	Coding task	Write a function, explain time complexity
DSA	Problem + explanation	Choose and justify an algorithm
ML	Concept + mini task	Explain overfitting and train a simple model
Deep learning	Concept + implementation	Build a small neural network
Software engineering	Design task	Design an API and explain trade-offs

10. Project Engine

The project engine should move from small projects to a portfolio-level capstone.

- Mini project: reinforces one newly learned concept.
- Integration project: combines several skills from a stage.
- Portfolio project: demonstrates the target skill to an employer or community.

- Hackathon-ready project: converts the learner's current skills into a practical challenge with a clear problem statement.

For example, a learner pursuing ML Engineering might receive: a data-cleaning project → a classification project → an API deployment project → a full ML application. A learner pursuing software engineering might receive: CLI project → REST API → database-backed application → deployed full-stack system.

11. Opportunity Matching

TechPath should not simply say “here are hackathons.” It should explain the match.

Example: A learner has completed Python, basic ML and one small project. Instead of recommending an advanced robotics hackathon, the system might recommend a beginner-friendly AI/data hackathon whose task requires the exact skills they are currently building.

Each opportunity record can include: title, organizer, type, deadline, eligibility, location/remote status, required skills, difficulty estimate, application URL, source URL and why it matches the learner.

12. Actor Definition

Recommended name: TechPath Learning Intelligence

Short description: Build personalized technical learning paths from web resources, assessments, projects and real-world opportunities.

Suggested slug: techpath-learning-intelligence

This name makes the product sound like a reusable data/automation tool rather than a consumer-only education app.

Apify Actors accept structured input and can produce structured output; they can also be called through the API, scheduled, and composed with other Actors. [\[cite\turn0search0\turn0search4\]](#)

13. Actor Input Schema

```
{ "goal": "Machine Learning Engineer", "currentLevel": "beginner", "knownSkills": ["Python basics"],  
  "hoursPerWeek": 8, "location": "Nigeria", "interests": ["NLP", "healthcare AI"], "learningPreferences":  
  ["video", "hands-on"], "depth": "deep" }
```

Optional inputs can include preferred language, free/paid resource preference, deadline, assessment difficulty and specific target roles.

14. Actor Output Schema

```
{ "profile": {...}, "targetRole": "Machine Learning Engineer", "skillGaps": [...], "roadmap": [...],  
  "resources": [...], "assessments": [...], "projects": [...], "opportunities": [...], "capstone": {...},  
  "sources": [...] }
```

Apify Datasets are designed for structured rows of extracted/processed data and can be exported in formats such as JSON, CSV and Excel. Key-value storage can hold less structured files or artifacts.

[\[cite\turn0search5\turn0search14\]](#)

15. Apify Tools / Actors / APIs

Component	Role in TechPath	Why
Our TechPath Actor	Main orchestrator	Accepts learner profile, runs discovery/analysis, returns structured roadmap.
Google Search Results Scraper	Web discovery	Finds candidate learning pages, tutorials, projects and opportunities from search queries. A current community Actor returns structured SERP data. [cite\turn1search0]
Apify Web Scraper	Page crawling/extraction	Extracts structured information from discovered pages. Apify's Web Scraper supports arbitrary websites and recursive crawling. [cite\turn1search1]
YouTube Scraper	Video resource discovery	Can retrieve YouTube metadata and transcripts where available; a current community Actor exposes these fields. [cite\turn1search2]
Apify Dataset	Structured storage	Stores resources, skills, opportunities and assessments as structured records.
Apify API	Integration	Lets other apps call TechPath and retrieve results.

Apify Scheduler	Freshness	Allows periodic re-runs so resource and opportunity data can be refreshed. [cite]turn0search2
Actor-to-Actor calls	Composition	Lets the main Actor combine specialized Actors into one workflow. [cite]turn0search0

Important implementation note

We should not build five separate Actors for the hackathon just to list five tools. The TechPath Learning Intelligence Actor should be the product. Existing Apify Actors are components it can call for discovery and extraction. This keeps the demo focused and makes the reusable value obvious.

16. Technical Workflow

- 1 Receive learner profile and target role.
- 2 Translate the goal into search intents: foundations, core skills, advanced skills, projects and opportunities.
- 3 Run web discovery using search-oriented Actors.
- 4 Crawl candidate pages and extract titles, descriptions, topics, prerequisites, difficulty and source metadata.
- 5 Normalize resources and remove duplicates.
- 6 Use an LLM to classify resources and infer likely prerequisites with confidence labels.
- 7 Build a skill graph: prerequisite → skill → advanced skill.
- 8 Compare the learner's known skills against the graph to identify gaps.
- 9 Generate assessments and project recommendations for the highest-priority gaps.
- 10 Search for practical opportunities that match the learner's current skills and target.
- 11 Return structured output in an Apify Dataset and output schema.

17. Example: Machine Learning Engineer

Input: beginner, Python basics, 8 hours/week, Nigeria, interested in NLP.

Possible output:

- Foundation review: Python, Git, NumPy, basic statistics.
- Core ML: supervised learning, evaluation, feature engineering, scikit-learn.
- Assessment: implement and evaluate a classification model.
- Project: build an end-to-end prediction system.
- Next: neural networks, PyTorch and deep learning.
- Advanced: NLP, embeddings and transformers.
- Later: RAG/agents only after the prerequisite layers are satisfied.
- Opportunity: recommend an appropriate AI/data hackathon based on current skills, not simply the most popular event.

18. Capstone Project Generator

At the end of a learning stage, TechPath can generate a structured capstone brief.

Field	Generated content
Problem statement	A real-world problem appropriate to the learner's current skill level.
Objective	What the learner should demonstrate.
Skills tested	Specific skills from the current roadmap.
Deliverables	Code, README, demo, report, tests, deployment, etc.
Milestones	Small steps from setup to final demo.
Evaluation	Rubric for correctness, engineering quality and explanation.
Extension	Optional advanced feature for the next skill level.

19. Hackathon MVP

The first version should be deliberately narrow. Trying to support every tech career, every website and every assessment format would make the prototype unreliable.

MVP target: one Actor that supports 3–4 technical tracks, with Machine Learning Engineering as the strongest demo path.

- Software Engineering
- Machine Learning Engineering
- Data Science
- AI Engineering

For the demo, we can run one learner profile through the Actor and show the complete chain: goal → web discovery → skill graph → gap analysis → resources → assessment → project → matching opportunity.

Apify's documentation recommends well-defined inputs and outputs for Actors, and its Academy notes that Actors are particularly suited to repeatable background automation rather than only consumer-facing interfaces. [\[cite\]turn0search1\[turn0search5\]](#)

20. What Success Looks Like

- A learner can enter a target role and receive a coherent first roadmap.
- The system explains why a topic is recommended before another topic.
- The learner receives at least one assessment for each major stage.
- The learner receives projects that directly reinforce the roadmap.
- The learner receives practical opportunities that match the skills being developed.
- The output is structured enough for another application or AI agent to consume.
- The same Actor can be reused for a different learner or technical career without changing its core code.

21. Risks and How We Handle Them

Risk	Mitigation
Bad or shallow resources	Use source quality signals, multiple sources and explicit source URLs.
Wrong prerequisite inference	Use confidence labels and allow human-readable reasoning rather than claiming absolute correctness.
Too much content	Return a small ranked set rather than every result found.
Outdated opportunities	Include dates and schedule periodic refreshes; flag stale records.
Overly broad product	Start with 3–4 technical tracks and one strong demo path.

22. Hackathon Positioning

Do not pitch this as “an AI learning app.” Pitch it as a reusable web-intelligence Actor.

The strongest framing is: “The internet has the content. TechPath builds the structure.”

Apify is useful because the learning ecosystem is constantly changing. New tutorials appear, old videos become outdated, new documentation is released, hackathons open and close, and project examples change. A scheduled Actor can revisit the web and refresh its dataset rather than relying on a static manually curated list. Apify supports scheduled Actor runs and reusable Actor-to-Actor workflows.

`[cite]turn0search2[turn0search0]`

23. Concept Note for the Application

TechPath is an Apify Actor designed to solve a problem I have personally experienced while learning technology: there is no shortage of information, but there is often no clear path connecting it. A learner can find Python tutorials, DSA problems, machine-learning courses, YouTube playlists, documentation, GitHub projects and trending topics such as LLMs and agents, yet still not know what should come first, what they are missing, or how to know when they are ready to move forward.

TechPath turns this fragmented web into structured learning intelligence. A learner provides a target role, current skills, experience level and learning constraints. The Actor discovers relevant resources, crawls and extracts their content, identifies the skills and prerequisites they cover, builds a dependency-aware roadmap, highlights skill gaps, generates short assessments, recommends projects and finds practical opportunities such as hackathons or internships that match the learner's current stage.

The project is not another course marketplace. The Actor is the reusable infrastructure behind a future learning assistant, dashboard or AI agent. Its output is structured data that other applications can consume through the Apify platform and API. The first prototype will focus on Software Engineering, Machine Learning Engineering, Data Science and AI Engineering, with Machine Learning as the main demonstration path.

24. 150-250 Word Problem + Apify Actor Answer Draft

Technical learning content is abundant, but the learning journey is fragmented. A beginner can find hundreds of courses, YouTube videos, documentation pages, GitHub projects and trending topics, yet still struggle to understand what should be learned first, what prerequisites are missing, and how to move from watching tutorials to actually proving a skill. This is especially difficult for learners who do not have a mentor to guide their progression. The same problem appears when learners look for hackathons, internships and practical opportunities: relevant opportunities are scattered across different websites and are rarely matched to a learner's current skill level.

TechPath Learning Intelligence will address this by using an Apify Actor to collect and organize the web into a structured learning pathway. The Actor will accept a learner's target role, current skills and preferences, discover relevant resources, crawl and analyze their content, identify skills and prerequisites, and build a staged roadmap. It will also generate assessments, recommend projects and discover practical opportunities that fit the learner's current growth stage. The result will be returned

as structured data through an Apify Dataset and API so it can be used by learning platforms, AI agents or other developer applications. Rather than creating another course website, TechPath turns the existing web into reusable learning intelligence.

25. 150-250 Word “What Are You Building?” Draft

We are building TechPath Learning Intelligence, a reusable Apify Actor that turns scattered technical-learning content into a structured, personalized development path. The user provides a target such as Machine Learning Engineer or Software Engineer, their current skills, experience level and available learning time. TechPath then searches the web for relevant courses, YouTube resources, documentation, project tutorials and other learning materials. It analyzes those resources to identify the skills they teach, their likely prerequisites and their relevance to the learner's target.

The Actor produces more than a list of links. It builds a staged roadmap, identifies skill gaps, creates short assessments to test readiness, recommends practical projects and finds hackathons, internships or other opportunities that match the skills the learner is currently developing. For the final stage of a pathway, it can generate a capstone project brief with a problem statement, objectives, required skills, milestones and evaluation criteria.

The first prototype will focus on Software Engineering, Machine Learning Engineering, Data Science and AI Engineering, with Machine Learning as our main demonstration. The core product is the Actor itself; a future frontend can use its structured output to create a learning assistant, dashboard or AI career coach.

26. 100-200 Word Apify Implementation Draft

TechPath will be built as a main Apify Actor that orchestrates web discovery, extraction, analysis and structured output. The Actor will accept JSON input containing the learner's target role, current skills, experience level, location and learning preferences. For discovery, we plan to use an Apify Google Search Results Scraper Actor to find candidate learning resources and opportunities, then use Apify's Web Scraper or another suitable crawler to extract content from the selected pages. A YouTube scraping Actor can be used where video metadata or transcripts are needed.

The TechPath Actor will normalize the collected information, remove duplicates, classify resources by skill and difficulty, infer prerequisite relationships, and generate the learner's roadmap, assessments and project recommendations. Results will be stored in an Apify Dataset as structured records. The Apify API will allow other applications to invoke the Actor and retrieve its output, while Apify Schedules can refresh time-sensitive resources and opportunities. Actor-to-Actor calls will keep specialized web collection tasks separate from the main learning-intelligence logic.

27. The Core Demo

For the hackathon demo, show one learner journey rather than a giant platform.

Demo prompt: “I am a beginner in Nigeria. I know basic Python and want to become a Machine Learning Engineer. I have 8 hours per week and I am interested in healthcare AI.”

Then show the Actor producing:

- 1 A short diagnostic/gap analysis.
- 2 A staged roadmap from foundations to ML engineering.
- 3 3-5 carefully selected resources for the current stage.

- 4 A short assessment.
- 5 A project that reinforces the current stage.
- 6 A more advanced capstone idea aligned with healthcare AI.
- 7 A relevant hackathon/opportunity matched to the learner's current skill level.
- 8 Source URLs and freshness metadata for traceability.

28. Final Product Definition

TechPath Learning Intelligence is a reusable Apify Actor that answers one question: “Given where I am and where I want to go in tech, what should I learn next, how do I prove I learned it, what should I build, and what real opportunity can help me grow?”

It uses the web as its raw material, Apify as its automation and data layer, AI as its reasoning/structuring layer, and the Actor interface/API as the reusable product.

Recommended positioning: “Not another course platform. A learning-path intelligence engine built on the web.”

29. Sources & Apify References

Apify documentation confirms that Actors are serverless cloud programs that accept structured input, perform a task and optionally produce structured output; they can be run manually, via API/CLI, scheduled and composed with other Actors. [cite\turn0search0\turn0search4]

Apify's Actor-ideas guidance says Actors are particularly suitable for isolated, repeatable backend automation jobs, including web crawling, data processing and AI-agent workflows. [cite\turn0search1]

Apify's current Store includes web scrapers and community Actors for search results and YouTube extraction; the specific community Actors should be rechecked immediately before implementation because Store listings and availability can change. [cite\turn1search0\turn1search1\turn1search2]

Apify supports Datasets for structured results and Schedules for recurring Actor runs. [cite\turn0search14\turn0search2]

This document is a working product blueprint. The Actor name, exact Store integrations and implementation choices should be finalized after checking the live Apify Store and hackathon rules immediately before coding.